

OS Security

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

System Security

- Security properties
 - Confidentiality
 - Integrity
 - Authentication
- Some resources (e.g., files) are sensitive
- How do we protect them from an attacker?
 - How do we prohibit the attacker from accessing those resources?

Access Control

- Access control is the selective restriction of access to resources according to a policy
- Fundamentals
 - Subject (Principal): the entity who is accessing the resource
 - E.g., a user, a process, ...
 - Object: a resource that can be accessed
 - E.g., a file, a device, a web page,
 - Policy: how subjects can access on objects
 - E.g., Alice cannot read file X

Access Control (Cont.)

- Authorization
 - Who should be able to perform which actions
- Authentication
 - Identify who is requesting to perform the action
- Audit
 - Log all actions attributed to a particular principal
- Accountability
 - Hold a principal accountable for actions he/she takes

Access Control Policy

- $\text{access}(S, O) = \text{true}$
 - Subject S is allowed to access object O
- Examples
 - $\text{access}(\text{Alice}, \text{Alice's home directory}) = \text{true}$
 - $\text{access}(\text{Alice}, /etc/passwd) = \text{false}$
 - $\text{access}(\text{Bob}, \text{Alice's home directory}) = \text{false}$
 - $\text{access}(\text{root}, /etc/passwd) = \text{true}$
 - $\text{access}(\text{Alice}, /share/bob/f.txt) = \text{true}$

Access Control Matrix

- The access control policies can be managed in a matrix

	Alice's home directory	Bob's home directory	/etc/passwd	/share/bob/f.txt	...
Alice	TRUE	FALSE	FALSE	TRUE	
Bob	FALSE	TRUE	FALSE	TRUE	
root	TRUE	TRUE	TRUE	TRUE	
...					

Permissions

- We can have finer-grained permissions on access to an object
 - Read, Write, Execute
- Examples
 - `access(Alice, Alice's home directory) = {read, write}`
 - `access(Alice, /etc/passwd) = {}`
 - `access(Bob, Alice's home directory) = {}`
 - `access(root, /etc/passwd) = {read, write}`
 - `access(Alice, /share/bob/f.txt) = {read}`

Access Control Matrix (Cont.)

	Alice's home directory	Bob's home directory	/etc/passwd	/share/bob/f.txt	...
Alice	{R, W}	{}	{}	{R}	
Bob	{}	{R, W}	{}	{R, W}	
root	{R, W}	{R, W}	{R, W}	{R, W}	
...					

Security Mechanisms

- There are two primary security mechanisms for enforcing access control policies
- Access Control List (ACL)
 - Each object has a list of who has what kind of access to it
- Capability-based Security
 - Each subject has an unforgeable token that grants access

Access Control List

- An object centric mechanism
- Every object has an ACL that defines what access permissions each subject can have on the object
- Each access is checked against the object's ACL
 - Identify the subject accessing it
 - Find the permission the subject has on the ACL
 - Make access control decision
- Each column of the access control matrix is an access control list
- Example: guest list

Capability-based Security

- A subject centric mechanism
- A capability defines a value that references an object with an associated set of access permissions
- Each access is checked against the capability
 - Subject presents the capability
 - The reference monitor checks if the capability has the access permission
 - Make access control decision
- Example: key
 - Another one?

Reference Monitor

- A reference monitor is used to mediate all accesses to objects
- Desired properties
 - Unbypassable
 - Tamper-proof
 - Verifiable
- Subjects cannot access objects directly
 - All accesses must go through the reference monitor, which checks if an access should be granted or not

Trusted Computing Base

- The Trusted Computing Base (TCB) is the component of the system that has to be correct for some security goal to be achieved
- All trusts are built upon the correctness of the TCB
- Ideally, TCB should be unbypassable, tamper-proof and verifiable
- Example
 - The TCB for enforcing file access control includes the OS kernel and file system drivers

DAC vs. MAC

- Discretionary Access Control
 - Object owners can set the access control policy for the objects they own
 - Example: Linux
- Mandatory Access Control
 - Access control policies for all objects are set by a centralized authority
 - Example: SELinux

Role Based Access Control

- Access control matrix can grow complex as number of subjects, objects, and possible operation grow
- Observation: Users change more often than roles

	/hr	/eng	/marketing	/all
Marketing	{}	{}	{R,W}	{R}
Executive	{R}	{R}	{R, W}	{R, W}
Engineering	{}	{R, W}	{}	{R}
HR	{R, W}	{}	{}	{R}

Attribute Based Access Control

- Assign attributes to subjects and/or objects
- Design security policies using attributes
- Access control matrix defines which attributes the subjects need to access objects with certain attributes
- Example
 - Role Based Access Control

UNIX Security Model

- Subjects
 - Users
- Objects
 - Directories
 - Files
 - Other types of files
 - sockets, pipes, hardware devices, ...
- Operations
 - Read, Write, Execute

UNIX Groups

- A UNIX user may belong to one or more groups
- The groups can be used to provide role based access control
- A user belongs to
 - A primary group (defined in /etc/passwd)
 - Several additional groups (defined in /etc/group)
- Use the command **groups** to check your groups

UNIX File Access Control

- Every file and directory has an owner and a simple ACL
- The ACL specify what role can perform what actions
 - Three roles
 - Owner, Group, Other
 - Three operations
 - Read, Write, Execute
- Permissions are set by the owner (or root)
 - Use command **chmod**
 - Cannot be delegated
- Example: rwx r-x r-x

Sticky Bit

- Can a user rename a file in a directory where the user has only write permission on the directory but not the file?
- Sticky bit
 - Off
 - If user has write permission on directory, can rename and delete files, even if not owner
 - On
 - Only file owner, directory owner, or root can rename or delete the file

UNIX Process IDs

- Each process has a real UID/GID, an effective UID/GID, and a saved UID/GID
 - **Real IDs**: represents the user who starts/owns the process
 - **Effective IDs**: used to determine if the process is “allowed to do things”
 - **Saved IDs**: used to drop and re-gain privileges
- If an executable file has the SUID bit set, when a process executes this program the process’s effective IDs are changed to the ones of the program owner

```
wei@wei ~ >>> ls -la /bin/su
-rwsr-xr-x 1 root root 40536 May 17 2017 /bin/su
```

Linux Capabilities

- Traditionally Unix distinguished between privileged processes (EUID == 0) and unprivileged processes (EUID != 0)
- Privileged processes are exempt from all kernel permission checks, while unprivileged processes are subject to all permission checks
- Capabilities can divide the power of superuser into pieces
 - If a program with several capabilities is compromised, the damage it can do to the system would be much less than the same program running with root privilege

Privilege

- To limit the impact of a security compromise
 - **Separating** privileged operations into a small separate TCB
 - The TCB must be verified for correctness
 - Any security bugs outside the TCB will not be catastrophic
 - **Isolating** untrusted components from the rest of the system (privileged components)
 - Give the least privilege required to perform tasks

Sandbox

- A sandbox is a mechanism that isolates a process from the rest of the environment
- If some (vulnerable) code is from an untrusted source or takes inputs from potential attackers, running in a sandbox aims to prevent the exploits from spreading outside of the sandbox
- Various sandbox designs exist, but typically the sandbox
 - Provides a controlled set of resources for guest programs to run in (e.g., memory, disk)
 - Do not allow (all) network access, file system access, other I/O

Android Process Isolation

- Android Application Sandbox
 - **Isolation**: Each application runs with its own UID in its own VM
 - Memory protection
 - Communication limited to using unix domain sockets
 - Only ping, zygote run as root
 - **Interaction**: Reference monitor checks permissions on inter-component communication
 - **Least Privilege**: Application announces permission
 - User grants access at install time or request time

Chrome Security Model

- The modern chrome browser uses a multi-process architecture
 - The main browser process
 - Manage user data, address bar, settings
 - Privileged operations, e.g., network, file I/O, ...
 - Several renderer processes
 - Render remote content in each tab
 - Restricted permissions
 - Run in a sandbox

Chrome Security Model (Cont.)

- The unprivileged renderer process can request privileged operations from the browser process
- If a renderer process is crashed (because of a remote attack)
 - The damage is limited to only this process
 - No other site data is at risk
 - The main browser process cannot be compromised either

Information Flow Tracking

- A security mechanism to track how information (or data) propagates as a system/program computes
- Objects are labeled with tags, denoting security classes
 - Confidential data
 - Downloaded from Internet, e.g., browser or mail
 - ...
- The tags are updated and verified along the computation
- Can help reason about confidentiality and integrity properties

Credits

- Some slides are based on the slides of David Wagner, and Zakir Durumeric